

Von Pygame-2D zu Ursina-3D

Typische Stolpersteine und wie man sie vermeidet

Eine verständliche Ergänzung zu PixelPortalRift – Phase 1

Für den ersten Wechsel von 2D-Denken zu 3D-Denken

Inhalt

1	1. Der wichtigste Unterschied: 2D gegen 3D	1
2	2. Stolperstein: Die dritte Koordinate vergessen	1
3	3. Stolperstein: Die Kamera ist nicht der Bildschirm	1
	3.1 Typische Symptome	2
4	4. Stolperstein: Pixel sind keine Welt-Einheiten	2
5	5. Stolperstein: Zeichnen und Entities verwechseln	2
6	6. Stolperstein: Ereignisse und Dauerbewegung sind verschieden	3
7	7. Stolperstein: Kollision ist nicht automatisch vorhanden	3
8	8. Stolperstein: 2D-Texturen auf 3D-Modelle setzen	4
9	9. Stolperstein: Klassen passen nicht automatisch zusammen	4
10	10. Eine gute Reihenfolge zum Debuggen	5
	10.1 Checkliste für Phase 1	5
11	11. Quellen	5

1. Der wichtigste Unterschied: 2D gegen 3D

In Pygame arbeitet das bisherige PixelPortalRift-Projekt hauptsächlich mit Bildschirmkoordinaten. Ein Punkt besteht im Kern aus x und y: x bedeutet links oder rechts, y bedeutet oben oder unten. Die sichtbare Welt wird auf eine flache Bildschirmfläche gezeichnet.

Ursina arbeitet dagegen mit Objekten in einer 3D-Szene. Ein Punkt hat normalerweise drei Koordinaten: x, y und z. x ist links/rechts, y ist oben/unten und z ist vorne/hinten. Außerdem schaut eine Kamera auf die Welt.

| Frage | Pygame 2D | Ursina 3D | | — | — | — | | Koordinaten | x, y | x, y, z | | Sichtbarkeit | direkt auf die Oberfläche zeichnen | Kamera sieht Entities | | Objekt | Bild, Rechteck oder Sprite | Entity mit Modell und Eigenschaften | | Schleife | selbst schreiben | meistens `app.run()` | | Bewegung | Bildschirmposition verändern | Position in der Welt verändern |

> **Merksatz:** In Pygame malst du häufig auf den Bildschirm. In Ursina stellst du Dinge in eine Welt und lässt die Kamera darauf schauen.

Die Pygame-Dokumentation zeigt eine selbst geschriebene Schleife mit Ereignissen, Zeichnen und `clock.tick(60)` [1]. Ursina beschreibt dagegen Entity als Grundobjekt der Szene und bietet eine automatisch aufgerufene `update()`-Methode [2].

2. Stolperstein: Die dritte Koordinate vergessen

Ein typischer Fehler ist, Pygame-Positionen direkt zu übernehmen. In Ursina kann zwar manchmal ein Zweier-Tupel funktionieren, bei 3D-Objekten solltest du aber bewusst mit drei Werten arbeiten.

```
# Falsch gedacht: nur Bildschirm-x und Bildschirm-y
position = (300, 200)

# Besser: Position in der 3D-Welt
position = (3, 1, 0)
```

Bei PixelPortalRift kann ein Boden zum Beispiel bei `y=0` liegen. Die Würfel werden über x und z verteilt. Die Höhe bleibt zunächst gleich:

```
for x in range(-5, 6):
    for z in range(-5, 6):
        Entity(model="cube", position=(x, 0, z))
```

Wenn du nur x und y benutzt, bewegst du ein Objekt möglicherweise nach oben, obwohl du eigentlich nach vorne laufen wolltest.

3. Stolperstein: Die Kamera ist nicht der Bildschirm

In Pygame entspricht die Bildschirmposition oft direkt dem Ort, an dem etwas gezeichnet wird. In Ursina besitzt die Welt eine Kamera. Ein Objekt kann korrekt platziert sein, aber trotzdem unsichtbar bleiben, wenn die Kamera in die falsche Richtung zeigt oder zu weit entfernt ist.

Zum Testen hilft eine sehr einfache Szene:

```

from ursina import *

app = Ursina()

cube = Entity(model="cube", color=color.orange, position=(0, 0, 0))

EditorCamera()

app.run()

```

Die Kamera darf am Anfang ruhig einfach bleiben. Ändere nicht gleichzeitig Spieler, Kamera und Welt. Erzeuge zuerst einen sichtbaren Würfel und verschiebe danach nur die Kamera.

3.1 Typische Symptome

| Beobachtung | Wahrscheinliche Ursache | Erste Prüfung | | — | — | — | | Nichts ist sichtbar | Kamera schaut nicht auf die Szene | Einen Würfel bei (0, 0, 0) testen | | Objekt ist winzig | 3D-Größen sind anders als Pixelgrößen | scale=1 oder scale=(1, 1, 1) testen | | Objekt ist hinter der Kamera | Falsche Z-Richtung | Kamera oder Objekt auf z verschieben | | Alles wirkt flach | Orthografische Kamera oder falsche Ansicht | Kameraeinstellung prüfen |

4 4. Stolperstein: Pixel sind keine Welt-Einheiten

In Pygame sind 300 meistens 300 Pixel. In Ursina ist 3 normalerweise eine Einheit in der Spielwelt. Ein Pygame-Spieler mit 32 Pixeln Breite wird deshalb nicht automatisch zu scale=32.

Starte mit kleinen Werten:

```

player = Entity(
    model="cube",
    scale=(1, 2, 1),
    position=(0, 1, 0),
)

```

Lege eine eigene Regel fest. Zum Beispiel kann ein Block genau eine Welt-Einheit breit sein. Dann ist der Spieler ungefähr einen Block breit und zwei Blöcke hoch. Diese Regel macht spätere Kollisionen und Weltgeneration einfacher.

5 5. Stolperstein: Zeichnen und Entities verwechseln

In Pygame wird häufig in jedem Frame der Bildschirm geleert und alles erneut gezeichnet. In Ursina erzeugst du eine Entity normalerweise einmal. Danach änderst du ihre Eigenschaften, zum Beispiel position, color oder enabled.

```

# Pygame-Idee: in jedem Bild neu zeichnen
# pygame.draw.rect(screen, ...)

```

```
# Ursina-Idee: einmal erzeugen, dann verändern
player = Entity(model="cube", color=color.azure)
player.x += 1 * time.dt
```

Wenn du in jeder `update()`-Runde neue Entities erzeugst, wächst die Szene immer weiter. Das führt zu schlechter Leistung und vielen scheinbar doppelten Objekten. Erzeuge dauerhafte Objekte außerhalb von `update()`.

6 6. Stolperstein: Ereignisse und Dauerbewegung sind verschieden

In Pygame unterscheidest du häufig zwischen `KEYDOWN` und abgefragten gehaltenen Tasten. In Ursina ist `input(key)` gut für eine einmalige Aktion. Für Bewegung während des Haltens ist `held_keys` in `update()` passender.

```
def input(key):
    if key == "space":
        print("Einmaliger Tastendruck")

def update():
    player.x += held_keys["d"] * 5 * time.dt
```

Ein häufiger Fehler ist, Bewegung nur in `input(key)` zu programmieren. Dann bewegt sich die Figur oft nur in einzelnen Sprüngen. Ein anderer Fehler ist, `time.dt` wegzulassen. Dann hängt die Geschwindigkeit von der Bildrate ab.

7 7. Stolperstein: Kollision ist nicht automatisch vorhanden

Ein Würfel ist sichtbar, aber er ist nicht automatisch eine feste Wand. Für Maus- oder Kollisionsabfragen braucht eine Entity einen passenden Collider. Die Ursina-Dokumentation nennt unter anderem `box`, `sphere`, `capsule` und `mesh` als Collider-Typen [3].

```
wall = Entity(
    model="cube",
    color=color.gray,
    position=(2, 1, 0),
    collider="box",
)
```

Der Collider beschreibt zunächst nur, **wo** eine Kollision erkannt werden kann. Er verhindert nicht automatisch jede Bewegung. Die eigentliche Regel musst du programmieren, zum Beispiel: Wenn vor dem Spieler eine Wand ist, darf sich der Spieler nicht weiterbewegen.

Im Pygame-Projekt übernimmt Player bereits wichtige Bewegungs- und Kollisionslogik. Beim Umbau solltest du diese Logik nicht blind kopieren. Prüfe zuerst, ob sie Pixelrechtecke benutzt und welche Teile auf Weltkoordinaten angepasst werden müssen.

8 8. Stolperstein: 2D-Texturen auf 3D-Modelle setzen

Ein Pygame-Bild ist eine Fläche auf dem Bildschirm. Eine Ursina-Textur wird auf ein Modell gelegt. Ein Würfel besitzt mehrere Seiten; deshalb kann ein einzelnes Bild anders wirken als ein Pygame-Sprite.

```
block = Entity(  
    model="cube",  
    texture="assets/blocks/stone.png",  
    scale=(1, 1, 1),  
)
```

Prüfe den Pfad genau. Ursina sucht Texturen relativ zum Projekt und zur verwendeten Asset-Struktur. Wenn das Bild nicht erscheint, teste zuerst einen farbigen Würfel ohne Textur. So weißt du, ob der Fehler bei der Szene oder beim Dateipfad liegt.

Für Pixel-Art kann außerdem eine zu starke Filterung das Bild unscharf erscheinen lassen. Das ist kein Logikfehler, sondern eine Darstellungseinstellung.

9 9. Stolperstein: Klassen passen nicht automatisch zusammen

Die alte Klasse Player aus Pygame kennt wahrscheinlich pygame.Rect, Pixelbreiten und Bildschirmgrenzen. Eine Ursina-Klasse erbt sinnvollerweise von Entity und arbeitet mit Positionen in der 3D-Welt.

```
class Player(Entity):  
    def __init__(self, position=(0, 1, 0)):  
        super().__init__(  
            model="cube",  
            color=color.azure,  
            scale=(1, 2, 1),  
            position=position,  
        )  
  
    def update(self):  
        self.x += held_keys["d"] * 5 * time.dt
```

Übertrage zuerst nur die Aufgabe der Klasse: Der Spieler soll sichtbar sein und sich bewegen. Inventar, Hunger, Werkzeuge und Animationen kommen erst danach. Kleine Klassen sind leichter zu testen als eine sofortige Komplettübersetzung.

10 10. Eine gute Reihenfolge zum Debuggen

Wenn etwas nicht funktioniert, ändere immer nur eine Sache auf einmal. Die folgende Reihenfolge spart Zeit:

1. Zuerst muss das Fenster starten.
2. Danach muss ein einzelner Würfel sichtbar sein.
3. Dann wird der Boden ergänzt.
4. Danach kommt der Spieler.
5. Erst dann kommen Bewegung und Kamera.
6. Kollisionen, Texturen, Inventar und Portale folgen zuletzt.

> **Gute Regel:** Wenn nach einer Änderung plötzlich nichts mehr sichtbar ist, gehe zum kleinsten funktionierenden Beispiel zurück und füge die letzte Änderung erneut, aber langsamer, hinzu.

10.1 Checkliste für Phase 1

| Frage | Ja/Nein | | — | — | | Startet python ursina_phase1.py ohne Fehlermeldung? | | | Ist mindestens ein Würfel sichtbar? | | | Werden x, y und z bewusst verwendet? | | | Wird Bewegung in update() mit time.dt ausgeführt? | | | Werden Entities nicht in jedem Frame neu erzeugt? | | | Ist klar, ob ein Problem von Kamera, Maßstab oder Textur kommt? | |

11 Quellen

[1 *Pygame Documentation*, Quick start und Referenz, abgerufen am 4. September 2026.]

[2 *Ursina Entity Basics*, offizielle Dokumentation zu Entities, Position, update() und Eingaben, abgerufen am 4. September 2026.]

[3 *Ursina API Reference*, offizielle API-Referenz zu Ursina, Entity und Collidern, abgerufen am 4. September 2026.]